

BBF-412-1 ATC DAY@2023

BASIC KNOWLEDGE OF PYTEST-BDD TEST FRAMEWORK

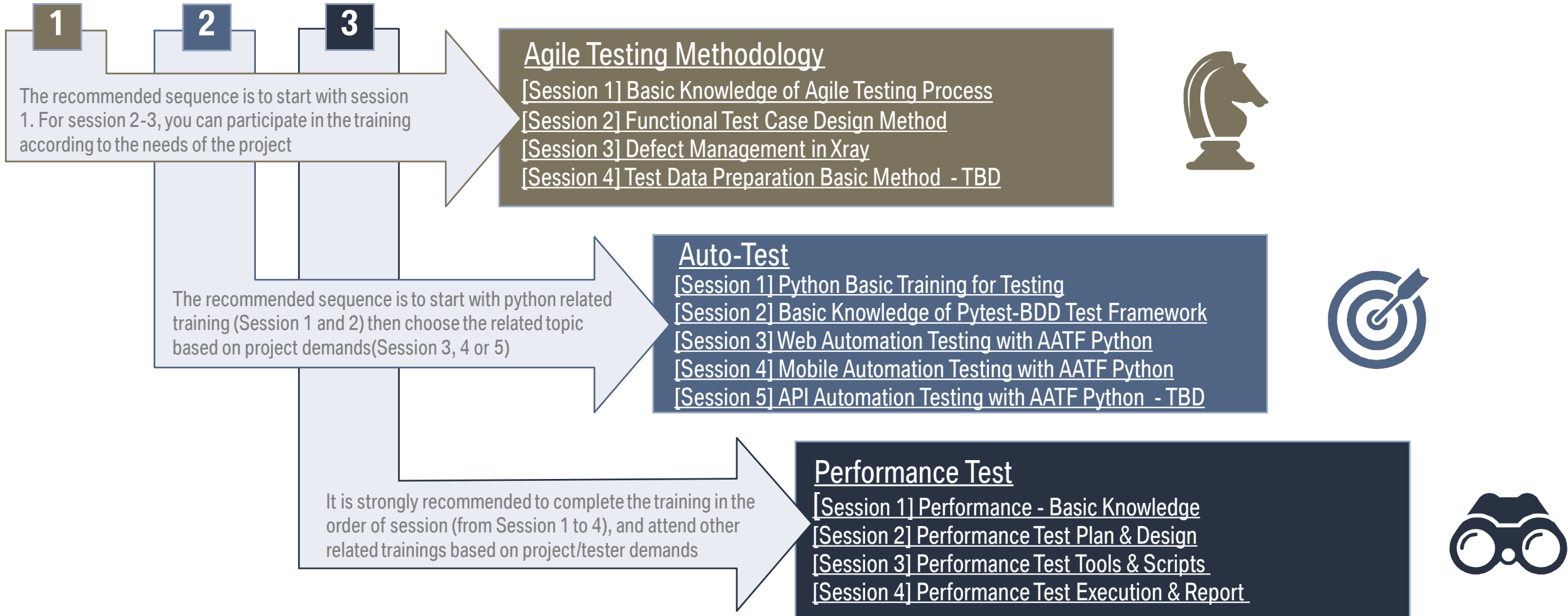
2023.03

BMW BRILLIANCE
AUTOMOTIVE



BACKGROUND

According to AWM is the Feature Team responsible of the generated quality of the product (including execution of functional / non-functional tests, according to specifications). Below is the Qualification Map Testing, it will help you to find the right training course, points out the mandatory training courses and shows the available training courses per competence area.



- **BASIC KNOWLEDGE OF PYTEST-BDD TEST FRAMEWORK**
 - What is Pytest-BDD
 - How to use Pytest-BDD
 - A Brief Introduction of AATF-Python

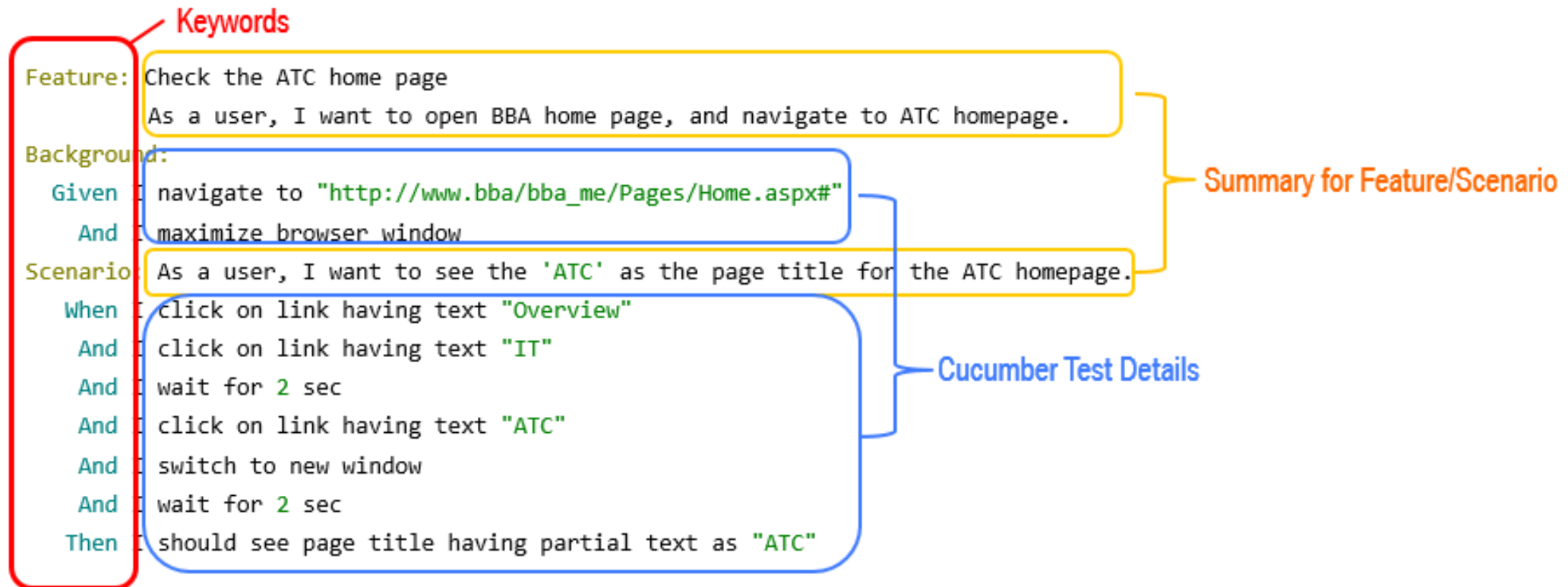
01

WHAT IS PYTEST-BDD

WHAT IS PYTEST-BDD

❖ BDD

- Behavioral Driven Development (BDD) is a software development **approach** that differs by being written in a shared language, which improves communication between tech and non-tech teams and stakeholders. In BDD approach, tests are written ahead of the code and more user-focused and based on the system's behavior.



KEYWORDS IN GHERKIN

The automation test script is written via Gherkin, it consist of several statements with special keywords:

Feature: Add to Cart

This feature will test functionality of adding different products to the User basket from different flow

Background: User is Logged In

Scenario: Search a product and add the first result/product to the User basket

Given User searched for Lenovo Laptop

When Add the first laptop that appears in the search result to the basket

Then User basket should display with 1 item

KEYWORDS IN GHERKIN

❖ Feature: Keyword

- Defines the logical test functionality you will test in this feature file
- Each Gherkin file begins with a Feature keyword

❖ Background: Keyword

- Defines steps which are common to all the tests in the feature file

❖ Scenario: Keyword

- Each Feature will contain some number of tests to test the feature, which is called a Scenario

Feature: *Add to Cart*

This feature will test functionality of adding different products to the User basket from different flow

Background: *User is Logged In*

Scenario: *Search a product and add the first result/product to the User basket*

KEYWORDS IN GHERKIN

❖ **Given: Keyword**

- Defines a precondition to the test

❖ **When Keyword**

- Defines the test action that will be executed

❖ **Then Keyword**

- Defines the Outcome of previous steps

❖ **And Keyword**

- Be used to add conditions to your steps
- Add more conditions to **Given**, **When** and **Then** statements

❖ **But Keyword**

- Be used to add negative type comments

Feature: Login Action Test

Description: This feature will test a Login and Logout functionality

Scenario: Unsuccessful Login with Invalid Credentials

Given User is on Home Page

When User Navigate to Login Page

And User enters Username and Password

But The user credentials are wrong

Then Message displayed Wrong Username & Password

WHAT IS PYTEST-BDD?

- ❖ Pytest-bdd is a BDD library for the Pytest runner which implements a subset of the **Gherkin language** to enable automating project requirements testing and to facilitate **behavioral driven development**.

```
pip install pytest-bdd
```

```
# content of publish_article.feature
```

```
Feature: Blog
```

```
    A site where you can publish your articles.
```

```
    Scenario: Publishing the article
```

```
        Given I'm an author user
```

```
        And I have an article
```

```
        When I go to the article page
```

```
        And I press the publish button
```

```
        Then I should not see the error message
```

```
        And the article should be published # Note: will query the database
```

! Note

Only "Given" steps should be used in "Background" section. Steps "When" and "Then" are prohibited, because their purposes are related to actions and consuming outcomes; that is in conflict with the aim of "Background" - to prepare the system for tests or "put the system in a known state" as "Given" does it. The statement above applies to strict Gherkin mode, which is enabled by default.

```
# content of test_publish_article.py
```

```
from pytest_bdd import scenario, given, when, then
```

```
@scenario('publish_article.feature', 'Publishing the article')
```

```
def test_publish():
```

```
    pass
```

```
@given("I'm an author user")
```

```
def author_user(auth, author):
```

```
    auth['user'] = author.user
```

```
@given("I have an article", target_fixture="article")
```

```
def article(author):
```

```
    return create_test_article(author=author)
```

```
@when("I go to the article page")
```

```
def go_to_article(article, browser):
```

```
    browser.visit(urljoin(browser.url, '/manage/articles/{0}/'.format(article.id)))
```

```
@when("I press the publish button")
```

```
def publish_article(browser):
```

```
    browser.find_by_css('button[name=publish]').first.click()
```

02

HOW TO USE PYTEST-BDD

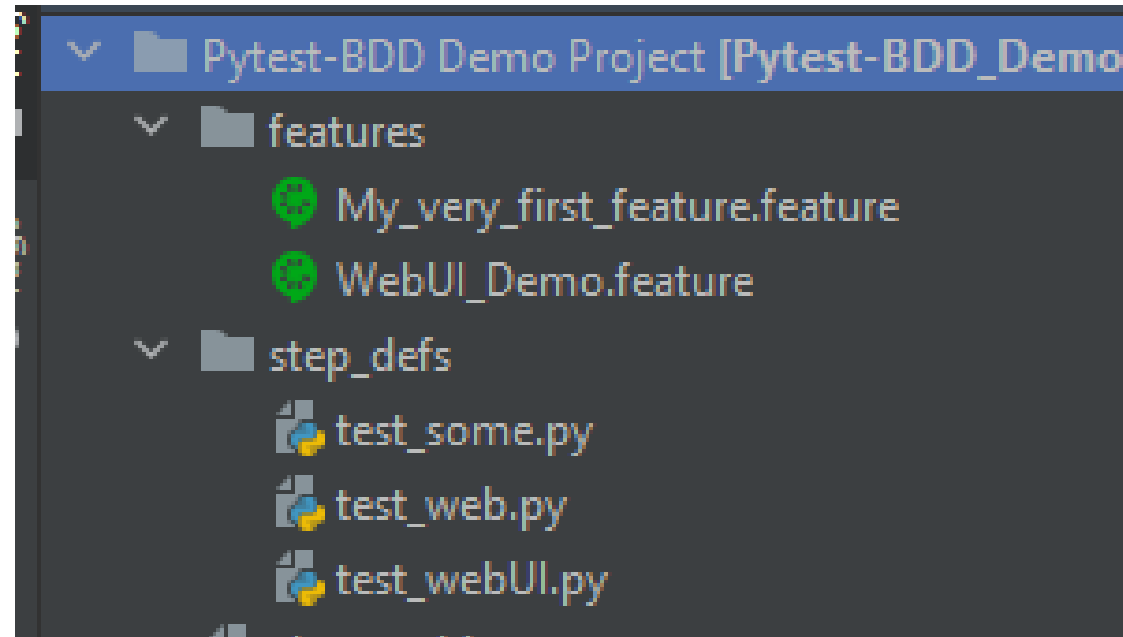
HOW TO USE PYTEST-BDD

1. Write Feature file
2. Define steps
3. Feature verify and debug
4. Run your tests and generate reports
5. Define reuse steps or functions in `conftest.py`

WRITE FEATURE FILE

- Create a python project
- Install related package: `pip install pytest-bdd`
- Tips on folder structure:

```
[project root directory]
|-- [product code packages]
|-- [test directories]
|   |-- features
|   |   |-- *.feature
|   |-- step_defs
|       |-- __init__.py
|       |-- conftest.py
|       |-- test_*.py
|-- [pytest.ini|tox.ini|setup.cfg]
```



SAMPLE FEATURE

Please write a feature to add two integers 123 and 456, and verify the sum

Feature:

This is a test feature

Scenario: Sample1

When there is an int var1 with value 123

And there is another int var2 with value 456

And I sum var1 with var2

Then I check the sum value should be 579

DEFINE STEPS

Tips:

For newcomers it's sometimes hard to write all needed test code without being frustrated. To simplify their life, a simple code generator was implemented. It allows to create fully functional (but of course empty) tests and step definitions for a given feature file.

pytest-bdd generate <feature file name> .. <feature file nameN>

Eg: pytest-bdd generate
features/My_very_first_feature.feature > step_defs/test_add.py

```
"""features\My_very_first_feature.feature.feature tests."""

from pytest_bdd import (
    given,
    scenario,
    then,
    when,
)

@scenario('features\My_very_first_feature.feature', 'Sample1')
def test_sample1():
    """Sample1."""

    @when('I sum var1 with var2')
    def _():
        """I sum var1 with var2."""
        raise NotImplementedError

    @when('there is an int var1 with value 123')
    def _():
        """there is an int var1 with value 123."""
        raise NotImplementedError
```

FEATURE VERIFY AND DEBUG

Use pytest command to run tests:

1. In general, pytest is invoked with the command `pytest`. This will execute all tests in all files whose names follow the form `test_*.py` or `*_test.py` in the current directory and its subdirectories. More generally, pytest follows standard test discovery rules.

```
pytest test_mod.py
```

Command-line Flags:

<https://docs.pytest.org/en/7.1.x/reference/reference.html#command-line-flags>

-s :表示输出调试信息，包括print打印的信息

-v :显示更详细的运行信息

-vs:这两个参数可一起使用

-k :匹配相应字符串的测试用例

-x :用例运行失败立即停止

:: :指定测试用例运行

2. Other ways of calling pytest:

You can invoke pytest from Python code directly:

```
retcode = pytest.main(["-x", "mytestdir"])
```


FEATURE VERIFY AND DEBUG

Add new steps to check minus function and debug

Feature:

This is a test feature

Scenario: Sample1

When there is an int var1 with value 123

And there is another int var2 with value 456

And I sum var1 with var2

Then I check the sum value should be 579

When I minus var2 to var1

Then I check the result should be 333

Tips:

pytest --generate-missing --feature features

STEP PARAMETERS

Provides a simple parser that replaces regular expressions for step parameters with a readable syntax like {param:Type}. The syntax is inspired by the Python builtin string.format() function. Step parameters must use the named fields syntax of pypi_parse in step definitions. The named fields are extracted, optionally type converted and then used as step function arguments. Supports type conversions by using type converters passed via extra_types

Eg: @when(parsers.parse('there is an int var1 with value {var1_value}'))

SAMPLE 2

Below is a sample of UI test. Let's see how to convert it into pytest-bdd

```
from selenium import webdriver
import time

from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service

def test_WebUI_Demo():
    #初始化浏览器并打开网页
    options = webdriver.ChromeOptions()
    options.add_argument('--incognito') # 无痕模式
    options.add_argument('--ignore-certificate-errors')
    options.add_experimental_option("excludeSwitches", ["enable-automation"])
    s = Service(r"../chromedriver.exe") # 指定driver路径
    driver = webdriver.Chrome(service=s, options=options)
    driver.get('https://demo.seleniumeasy.com/basic-first-form-demo.html')
    #窗口最大化
    driver.maximize_window()
    time.sleep(3)
    #输入内容
    driver.find_element(By.ID, 'sum1').send_keys(20)
    driver.find_element(By.ID, 'sum2').send_keys(20)
    #点击提交并截图
    driver.find_element(By.XPATH, "//*[@id='gettotal']/button").click()
    driver.execute_script("window.scrollTo(0,document.body.scrollHeight)")
    driver.get_screenshot_as_file('screen1.png')
    #验证结果
    var_sum = driver.find_element(By.ID, 'displayvalue').get_attribute('innerText')
    print(f"The sum value is: {var_sum}")
    assert int(var_sum) == 40
```

RUN YOUR TESTS AND GENERATE REPORTS

Use Allure to generate test reports:

Allure installation

1. Install allure-pytest-bdd in python : `pip install allure-pytest-bdd`
2. Download Allure from: <https://github.com/allure-framework/allure2/releases>
3. Config Allure in system environment variable PATH

Generate reports by using allure:

1. `pytest -vs test_some.py --alluredir=./report/json`
2. `allure generate report/json -o reports/html --clean`

DEFINE REUSE STEPS OR FUNCTIONS IN CONFTEST.PY

- It is possible to define some common steps in the parent conftest.py and simply expect them in the child test file.

content of common_steps.feature

Scenario: All steps are declared in the conftest

Given I have a bar

Then bar should have value "bar"

content of conftest.py

```
from pytest_bdd import given, then
```

```
@given("I have a bar", target_fixture="bar")
```

```
def bar():  
    return "bar"
```

```
@then('bar should have value "bar"')
```

```
def bar_is_bar(bar):  
    assert bar == "bar"
```

content of test_common.py

```
@scenario("common_steps.feature", "All steps are declared in the conftest")
```

```
def test_conftest():  
    pass
```



03

A BRIEF INTRODUCTION OF AATF-PYTHON

AATF JAVA VS AATF PYTHON (PYTEST-BDD)

	ATC Automation Testing Framework	Pytest- BDD Testing Framework
Programing Language	Java	Python
Environment Configuration	Easy , most of configurations is already preset	Need to install dependency libraries manually
Learning Cost	High	Low , Python is one of the most popular languages for software testers
Data Separation	No, data is written in feature file	Yes , data maintenance in YAML or excel is supported
User Guide	Rich user guides based on the user feedback in these years	Newborn, need more enhancement in future

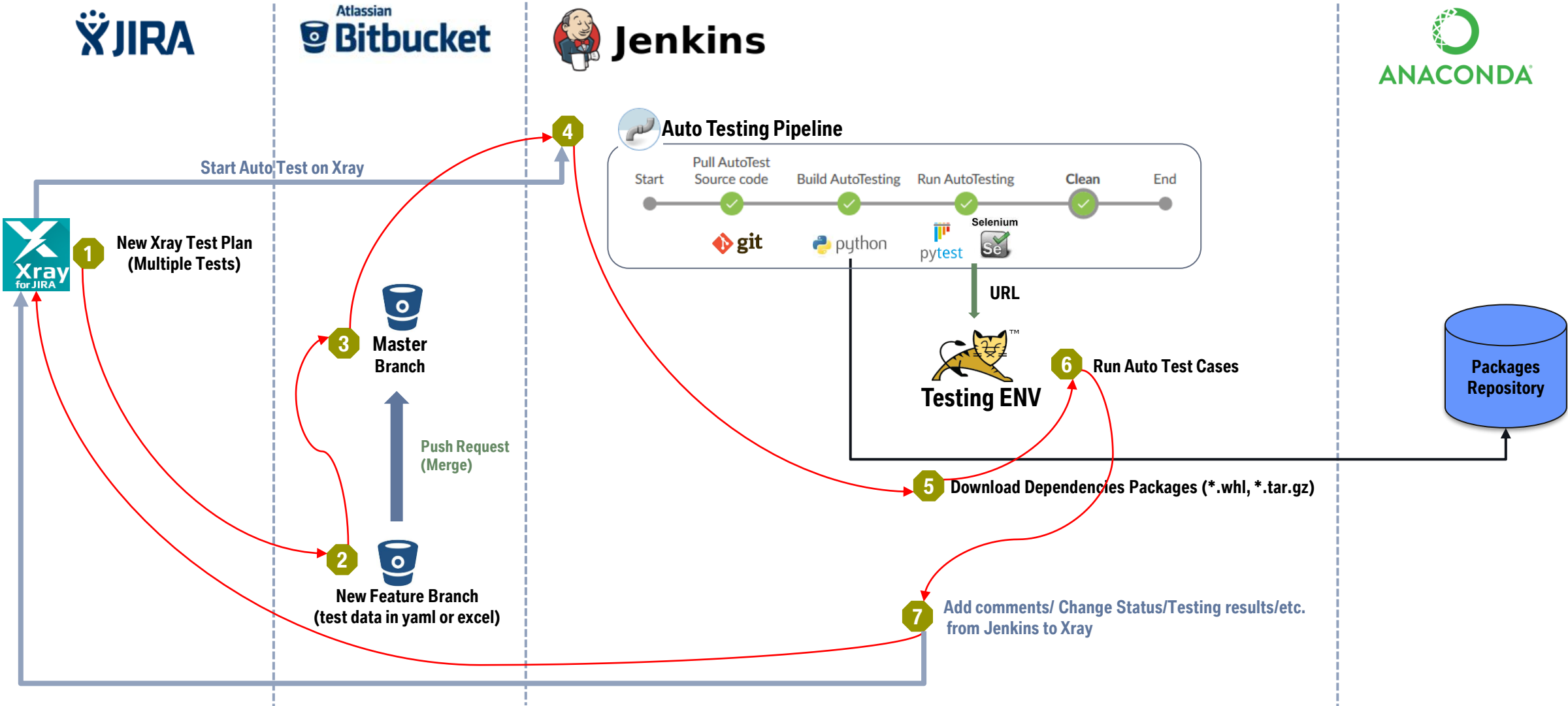
COMMON STATEMENTS FOR GUI

Statement	Function
Open browser and navigate to “https://.....”	Navigate to specified URL
I maximize browser window	Maximize browser
I click on element having xpath “//span[...]”	Click on web element using locator including index, id, name, class, xpath, css
I enter “....” into input field having id “....”	Enter text into input field using locator including index, id, name, class, xpath, css
I switch to frame having id “....”	Switch to frame by web element
I should see page title as “....”	Check the title
Element having id “....” should have text as “....”	Check element text using locator including index, id, name, class, xpath, css
I wait for ... sec	Wait for specific period of time
I take screenshot	Screen shot

COMMON STATEMENTS FOR API

Statement	Function
I set form data "{.....}"	Set the request body with either JSON, XML or HTML
I set the request header with name "...." and value "...."	Set a request header with specified name and value
I set auth with name "...." and password "...."	Set authentication with username and password
I send get/post request to "https://...."	Send HTTP Get/Post Request
I check the response HTTP status code equals "..."	Assert the response status code equals the specified value
I check the response including "...."	Assert the response body includes the specified char sequence

AUTOMATION TEST IN XRAY



AUTOMATION TEST IN XRAY

Create Issue

Configure Fields

Project*
ATC Testing Xray (ATX)

Issue Type*
Test

General Details Links Portfolio

Summary*

Description

Style B I U A

Create Issue

Configure Fields

General Details Links Portfolio

Steps

Step Data Expected Result

Add

Test Type
Cucumber

Choose the Test Type

Cucumber Type
Scenario

The Cucumber Scenario Type

Cucumber Scenario

Given I set the request header with key Content-Type and value application/json

And I set the request header with key client-id and value 113

And I set the request header with key third-session and value 53cb579469fb4e

ping 被禁用, 返回405

#Then I use method POST to ping the URL https://motorradapp.bmw.com.cn/BmwMc

发送GET请求到url, 并将结果保存入变量

Then I send request by GET method to https://motorradappoc.bmw.com.cn/BmwMc

#{"hasMore":true,"data":null,"status":200,"list":[{"id":39,"coverImg":"https

校验是否为null

Then I check the response JSON \$esncesData.status should not be null

校验是否为数字类型

Then I check the response JSON \$esncesData.status value should be number

校验状态值是否为200

Then I check the response JSON \$esncesData.status equals 200

Test Classification

None

End-2-End test

User Acceptance Test

Load & Performance Test

Interoperability test

Regression / Automation

Regression Relevant

Automation Relevant

AUTOMATION TEST IN XRAY

atc.bmw-brilliance.cn/jira/browse/ATX-1598

BBA 常用网站 测试网址 技术网站 temp 菜谱 自动化测试

Dashboards Projects Issues Boards Portfolio Tests Create Search

unavailable from 11:00 on Apr.4th to 19:00 on Apr.4th.

ATC Testing Xray / ATX-1598

MOTO demo

Edit Comment Assign

Details

Type: Test

Priority: Medium

Labels: APIProxy

Manual Test Steps:

This test is not associated with Test Plans yet.

Associate Test Plans

Test Runs

FILTERS

Project	Version (project dependent)	Status	Start
All Projects	Select a project to		DD-MM-YYYY

Show 10

Key	Fix Version/s	Revision	Executed By	Started	Finished
ATX-1599			Jenkins_Jira	45 minutes ago	45 minutes ago

atc.bmw-brilliance.cn/jira/secure/XrayExecuteTestIddefault.jspta?testExecIssueKey=ATX-1599&testIssueKey=ATX-1598

BBA 常用网站 测试网址 技术网站 temp 菜谱 自动化测试

ATC Testing Xray / Test Execution: ATX-1599 / Test: ATX-1598

MOTO demo

Import Execution Results Export to Cucumber Return to Test Execution

Context	Duration	Status
-	2 sec	PASS
Hooks		
Before bba.atc.autotest.glue.b.p.a(io.cucumber.java.Scenario)	2.000 ms	PASS
After bba.atc.autotest.glue.b.q.a(io.cucumber.java.Scenario)	-	PASS
Steps		
Given I set the request header with key Content-Type and value application/json	372.000 ms	PASS
And I set the request header with key client-id and value 113	-	PASS
And I set the request header with key third-session and value 53cb579469fb4e49af5dd53894f862cd	-	PASS
Then I send request by GET method to https://motorradappoc.bmw.com.cn/BmwMotor/wxapi/essence/esnces?pageSize=3&startIndex=0 and store the response as esncesData	1284.000 ms	PASS
Then I check the response JSON \$esncesData.status should not be null	346.000 ms	PASS
Then I check the response JSON \$esncesData.status value should be number	9.000 ms	PASS

#enjoyIT